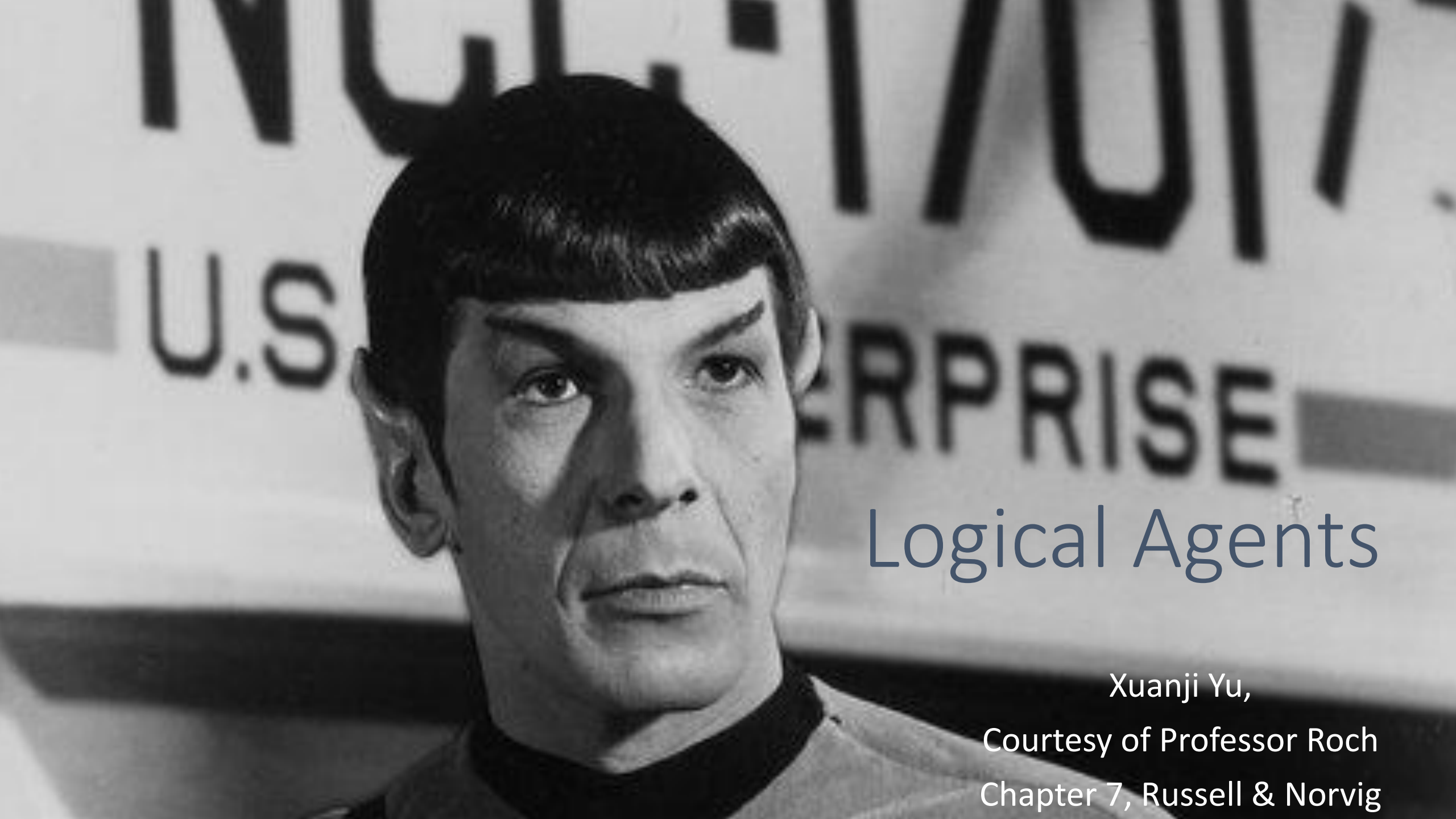


Logical Agents

Xuanji Yu,
Courtesy of Professor Roch

Chapter 7, Russell & Norvig

Logical agents

AI that thinks/acts using logical reasoning or in terms of true/false statements and rules like if-then statements.

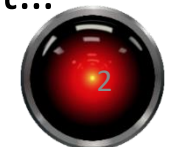
Two key components of logical agent

- Representation of the world (Knowledge)
  - Background information
  - Percepts
- Ability to logically draw conclusion (Reason)
  - Derive new information based on *inference*

Advantages: Explainable; Reliable; Flexible

Search algorithm generates successors and evaluate them but do not 'understand' much about the setting

- Example: is it possible for a chess player to have 8 pawns and 2 queens?
  - Search algorithm would search through tons of states to see if this ever happens, but…



Knowledge base (KB)

- Sentence – A statement about the agent's world that represent facts, rules, or condition

  - Based on sensors

    Danger around corner.

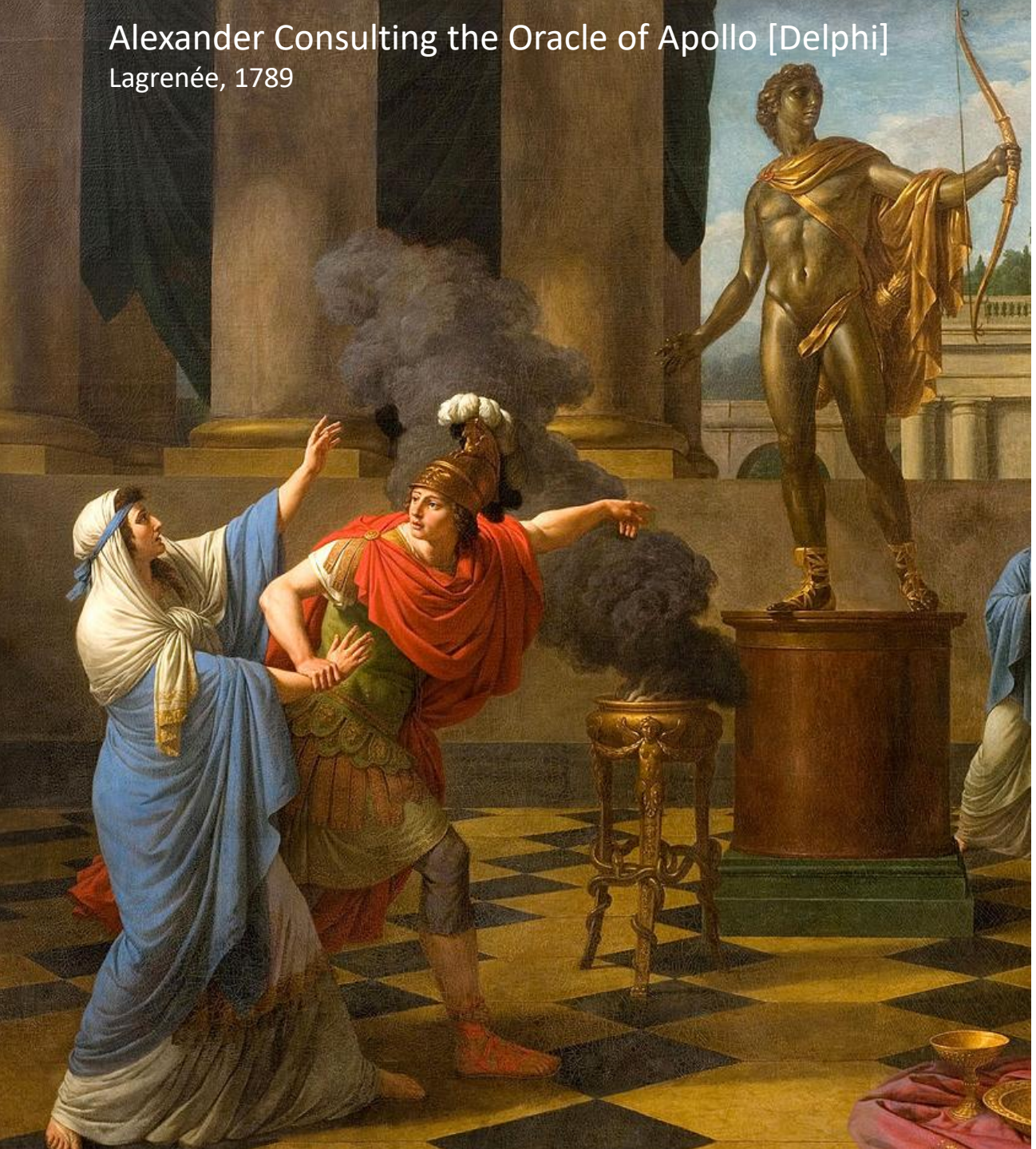
    Range to target is 50 m.

    Day is cloudy.

  - Based on background knowledge (or possibly learning)

    Solar cells lose efficiency on cloudy days.

image: invenergy.com


Alexander Consulting the Oracle of Apollo [Delphi]
Lagrenée, 1789

Knowledge base (KB) operations

Agent interactions with KB

- Tell – Agent informs KB of percepts
  Adds new *timestamped* sentences to the KB

- Ask – Agent asks KB which action should be taken. Reasoning occurs here.

- Tell – Agent tells KB that the action performed.

Knowledge base (KB) example

A medical diagnosis system

Sentences

- Rule: If (Fever AND Cough), then (Illness = Flue)
- Fact: Fever = True, Cough = True
- Inference: Illness = Flue

KB operation

- Tell the system: Tell(Fever = True), Tell(Cough = True)
- Ask the system: Ask(Illness = Flue)

KB operations

We will use the following functions to build sentences:

- Make-Percept-Sentence – Construct a sentence from a percept.
- Make-Action-Query – Construct sentence that asks which action should be done at current time.
- Make-Action-Sentence – Construct sentence indicating that the specified action was executed at time t.

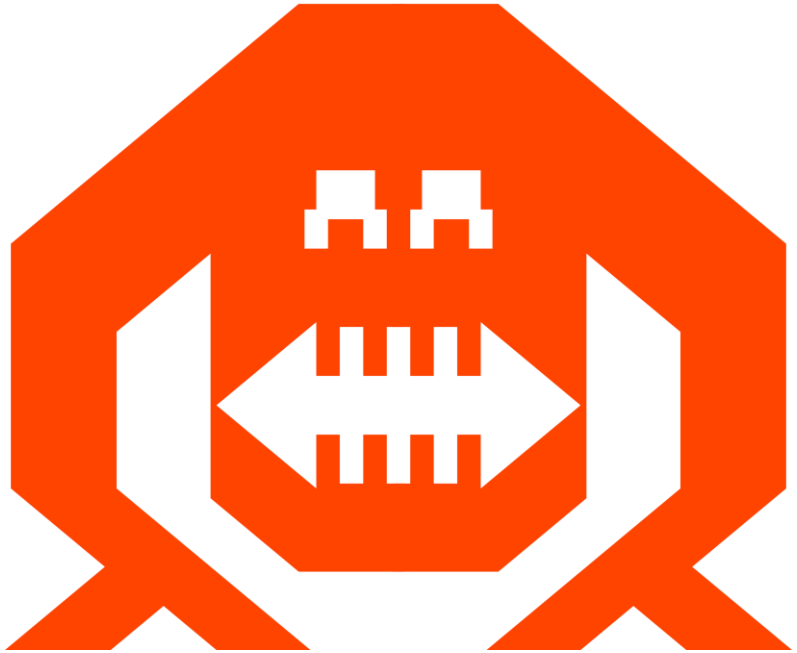
KB-agent

```
class KBagent:
   KB # knowledge base
   t = 0

   def getaction(percept):
      tell(KB, make-percept-sentence(percept, t))
      action = ask(KB, make-action-query(t))
      tell(KB, make-action-sentence(action, t))
      t = t+1
      return action # caller performs action
```

t could be a real timestamp instead of a counter

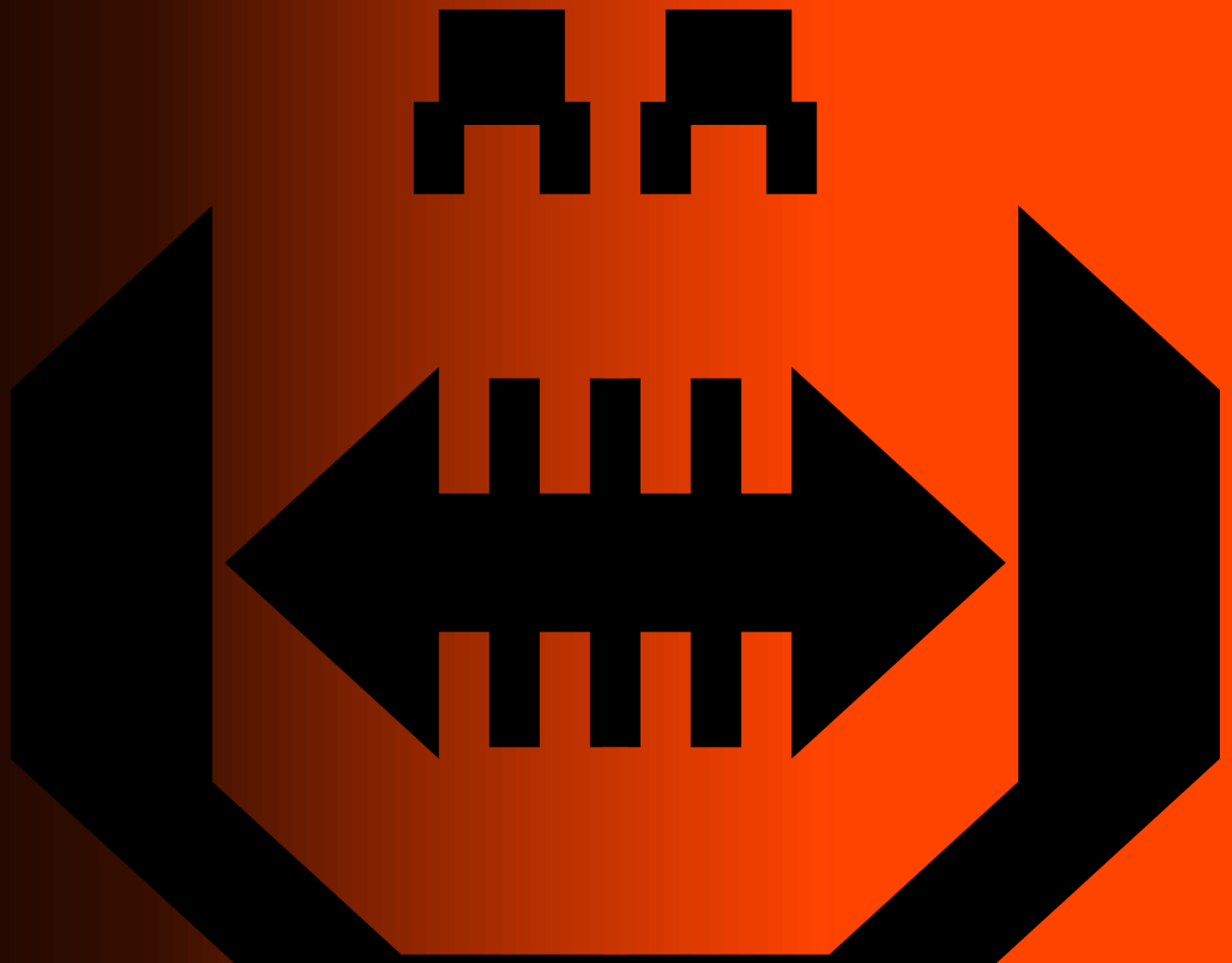
Beware the wumpus



- Early computer game
  - Maze of dark caverns with
    - Pits – too dark, you might fall in
    - Wumpus – Sedentary beast.
      Does not move but it will eat you alive if you stumble into it…
    - Pot of gold
  - Your goal
    - Armed with a bow and single arrow, climb down into the caverns, grab the gold and get out without falling into a pit or being gobbled up by the wumpus.
    - Try not to shoot the wumpus, he's an apex predator and good for the environment.

Beware the wumpus

- Actions
  - Move *forward*
  - Turn left
  - Turn right
  - Shoot (can only do this once) – Arrow flies in the direction you fire until it hits a wall or kills the wumpus.
  - Grab – Grab the gold if it is in your current position.
  - Climb – Climb out of the case, only valid in starting cave.



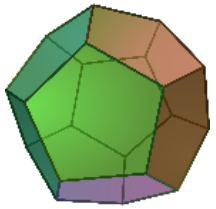
Sensors

- Chemical – Wumpus emits a *stench* that can be perceived in the next cavern
- Tactile
  - *breeze* is perceived when next to a pit
  - *bump* when agent walks into a wall
- Visual – All that glitters is gold, agent perceives *glitter* when in the same room as the gold.
- Auditory – The wumpus is usually pretty quiet, but emits a blood curdling *scream* when it dies.

Beware the wumpus

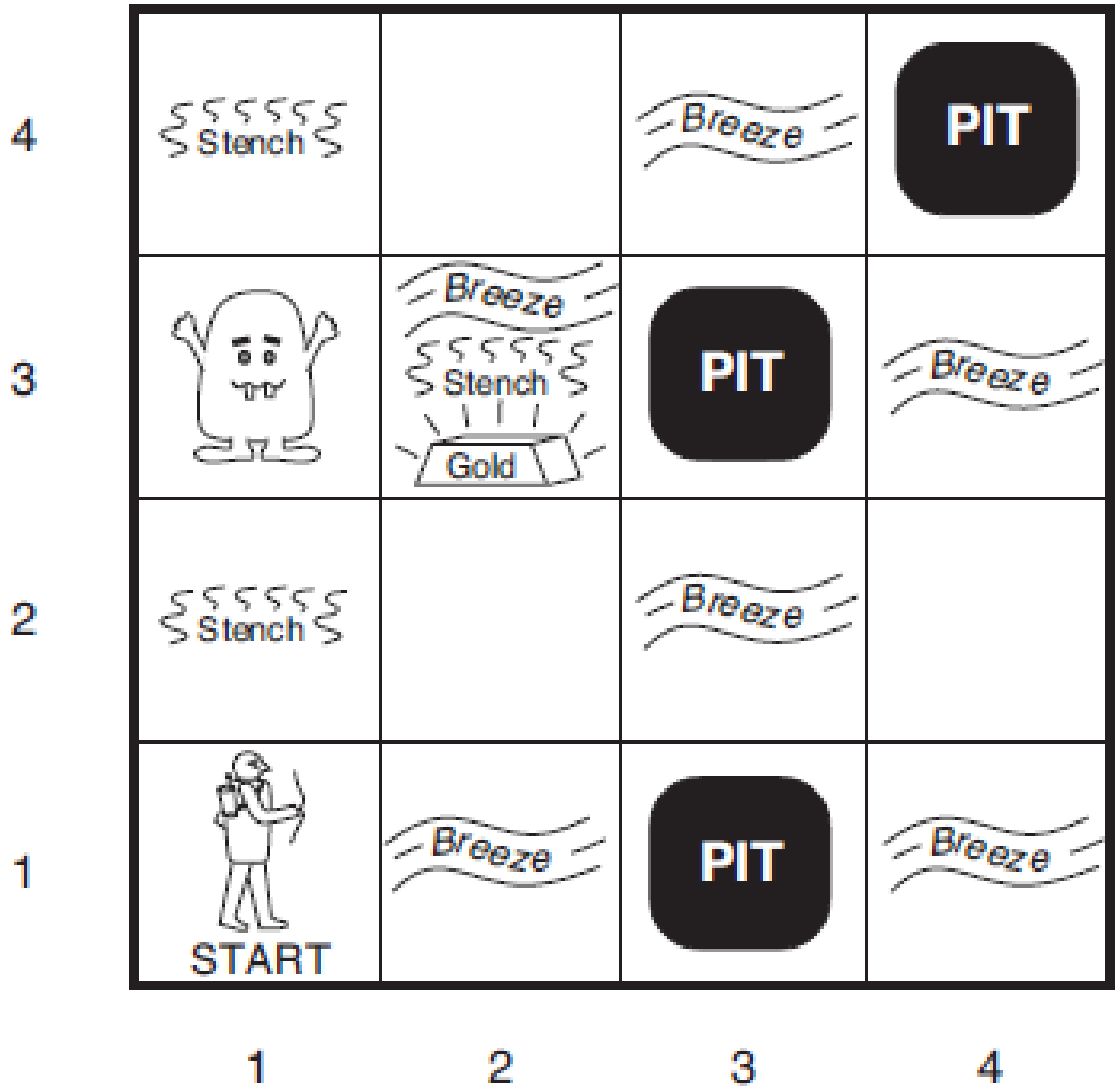
- Environment
  - 4x4 grid of caverns*
  - Grids other than the start square
    - P(pit) = .2
    - Wumpus placed randomly in non-pit, non-start cave
    - Gold randomly placed
  - Agent starts in x=1, y=1 facing such that they move positively along the y axis.

  - Some environments are unfair (e.g. gold in a pit or surrounded by pits ~ 21% of time)



*Written by Gregory Yob in the early 1970s, the cave topology was based on a dodecahedron, you can play a modified version online.

Welcome to wumpus world



Initial knowledge base

- Basic knowledge of environment (e.g. perceived a breeze next to a pit.
- Current location [1,1] is safe

- First percept:

  [No Stench, No Breeze, No Glitter, No Bump, No Scream]

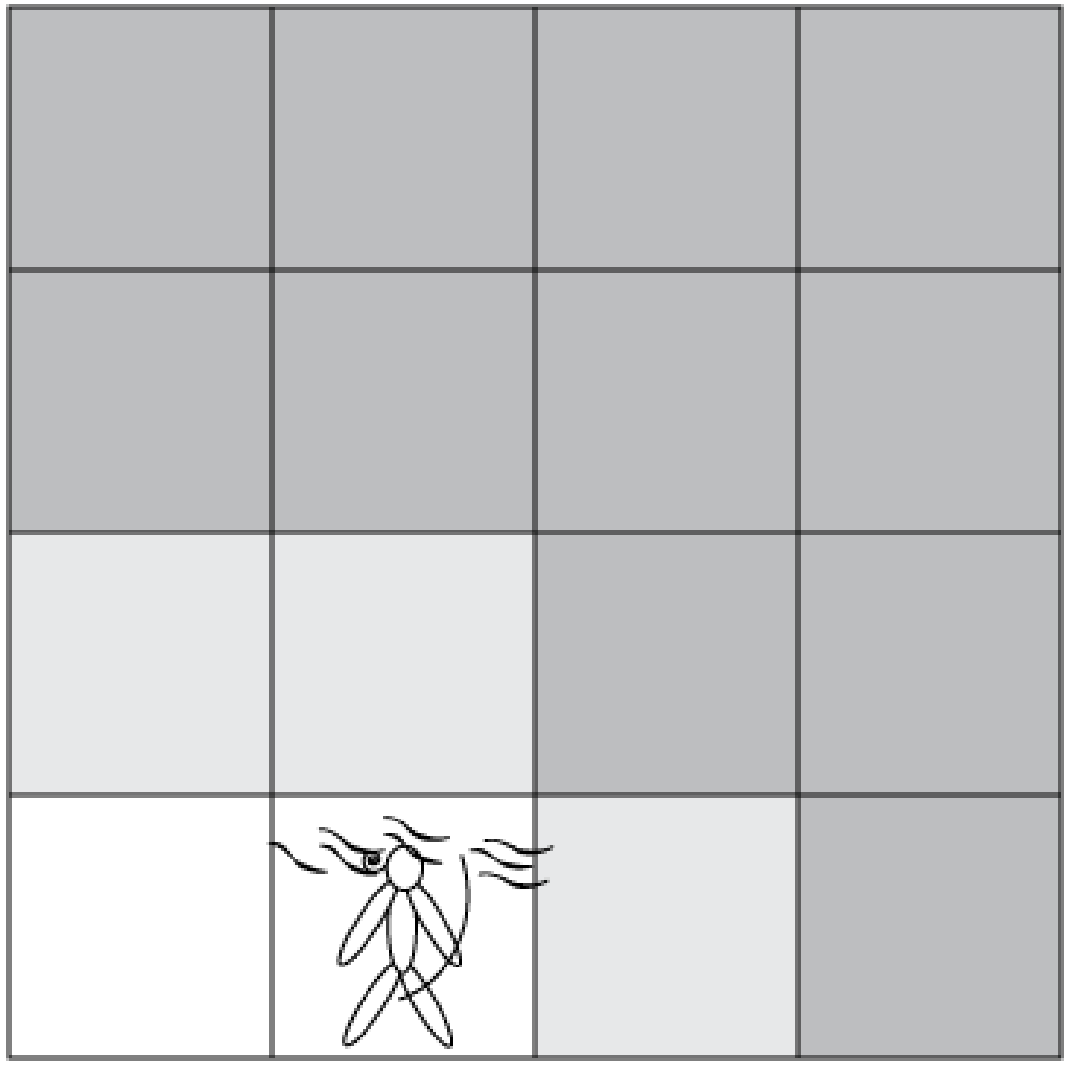
- What can we learn?

[No Stench, No Breeze, No Glitter, No Bump, No Scream]

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

A - agent position
OK - safe

Agent moves to 2,1



2,1 Percepts
No Stench
Breeze
No Glitter
No Bump
No Scream

White caves – visited
Light gray caves – surrounding visited ones
Dark gray caves – The great unknown

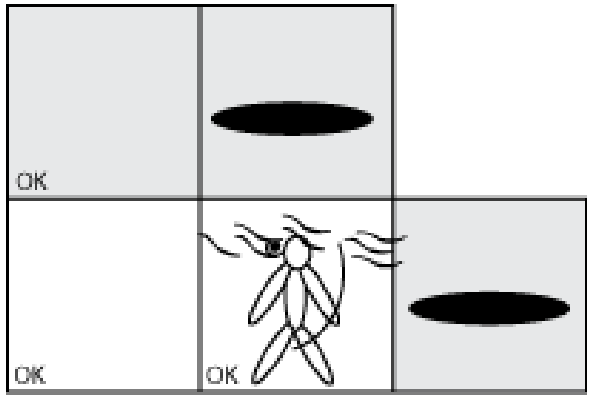
Danger approaches…

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

A - agent position
OK – safe
B – breeze
P? – possible pit

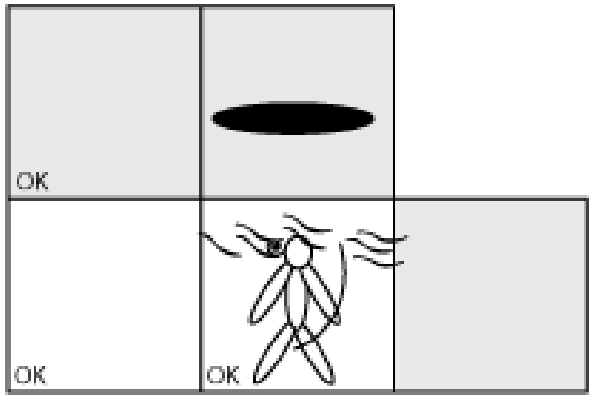
2,1 Percepts
No Stench
Breeze
No Glitter
No Bump
No Scream

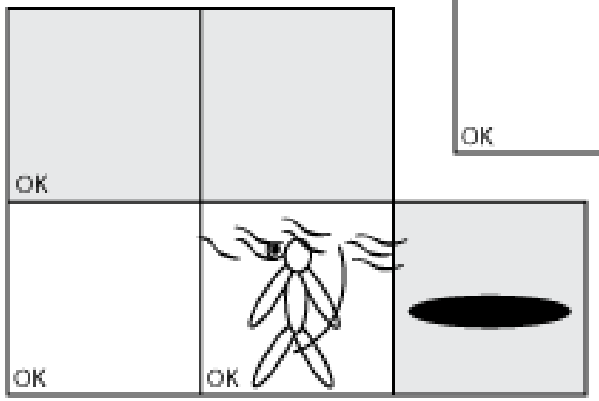
Knowledge base is extended based on percept breeze



Enumeration of instances

Three possible worlds



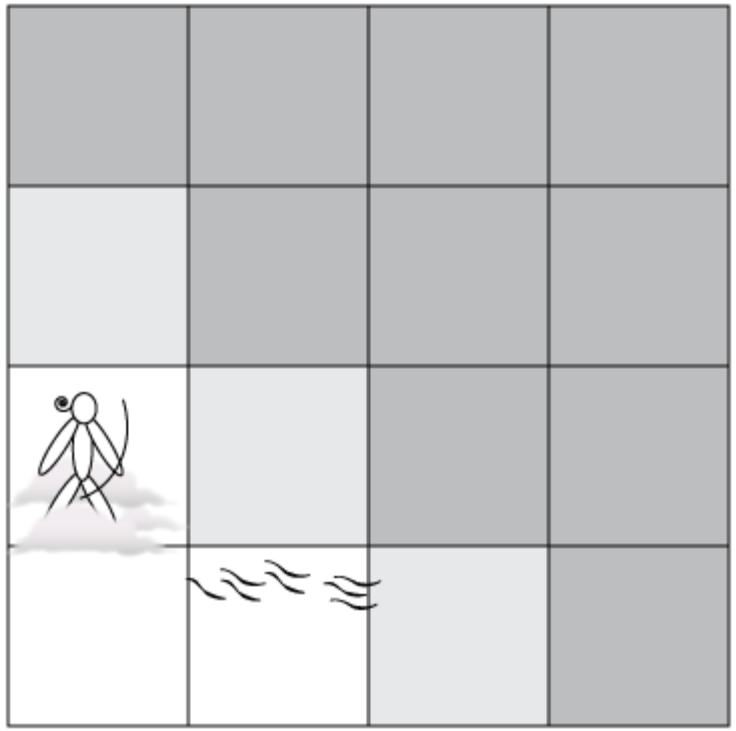


Playing it safe

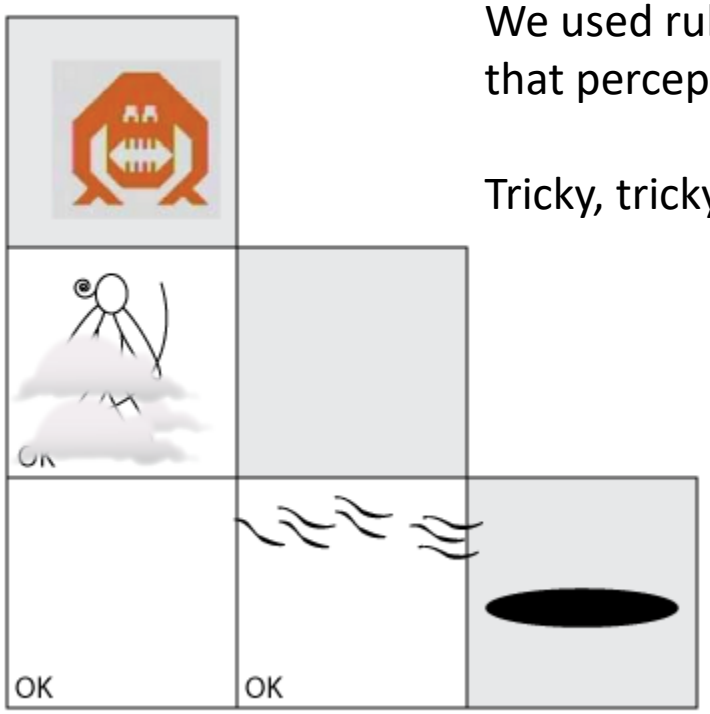
- We don’t know if it is safe to move into 3,1 or 2,2
- Move back to 1,1 then 1,2

I SMELL A WUMPUS!

- What can we learn based on our rules?



Logical deduction



We used rules to infer something that percepts did not tell us.

Tricky, tricky…

A little formality

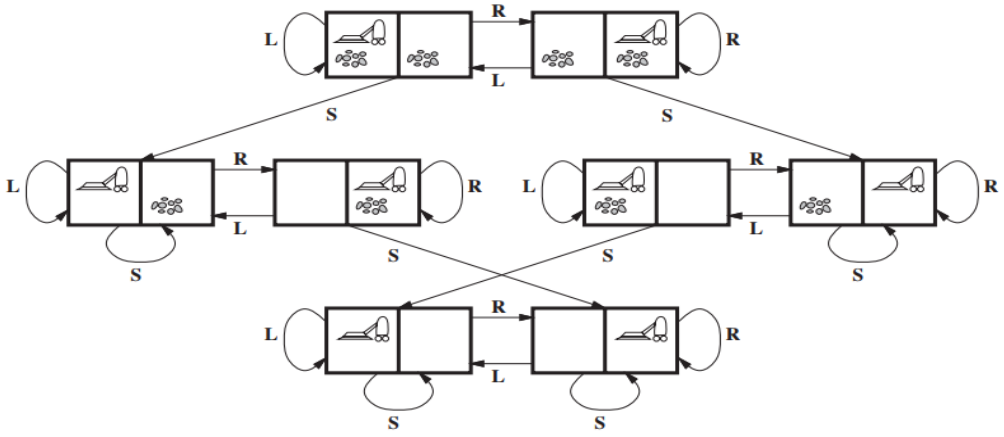


Figure 3.3 The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

- Models
  - Representation of world
  - Assignment of values to variables
  - All possible worlds consists of all possible assignments to model variables.
  - The knowledge base is some subset of the possible worlds.
- Sentences
  - Require syntax
  - Have semantics with respect to models.
    In most logical models: True/False

e.g. There is a pit at 3,1

Satisfaction and entailment

- Suppose a sentence α is true in model M, then we state:
  M satisfies α
  or equivalently: M is a model of α
- M(α) means all models that satisfy α.

- Reasoning – entailment
  - Sentence α entail the sentence β

$$\alpha \vDash \beta \text{ if and only if } M(\alpha) \subseteq M(\beta)$$

in every model in which α is true, β is also true
α is a *stronger* assertion* than β; there may be more worlds associated with β
That is: β logically follows from α if $\alpha \vDash \beta$

*stronger can be interpreted as tighter here

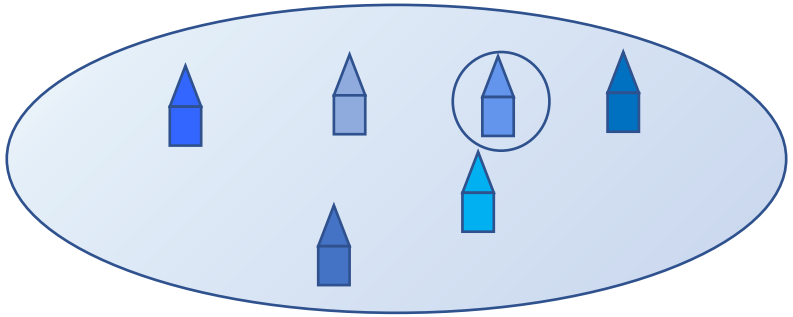
Entail vs Imply

Concept	Symbol	Meaning	Context
Imply	A $\Rightarrow$ B	If A, then B	Within logic formulas A can be false, A $\Rightarrow$ B is true
Entail	$\alpha \vDash \beta$	β follows from α	Across models KB $\vDash \beta$, KB always (or at least agent believes to be) true

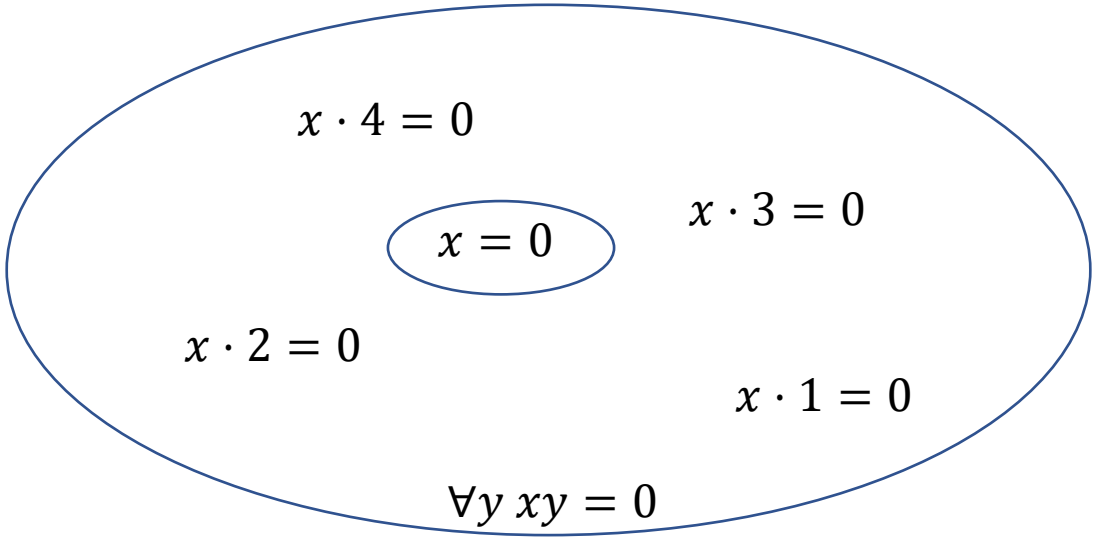
Imply ($\Rightarrow$) is used in the agent's rules:Rain $\Rightarrow$ Wet
Entail ($\vDash$) is used when reasoning: If the knowledge base contains Rain, then the agent can say KB $\vDash$ Wet — i.e., Wet is entailed.

Examples of entailment

- House is cornflower blue ⊨ house is a shade of blue



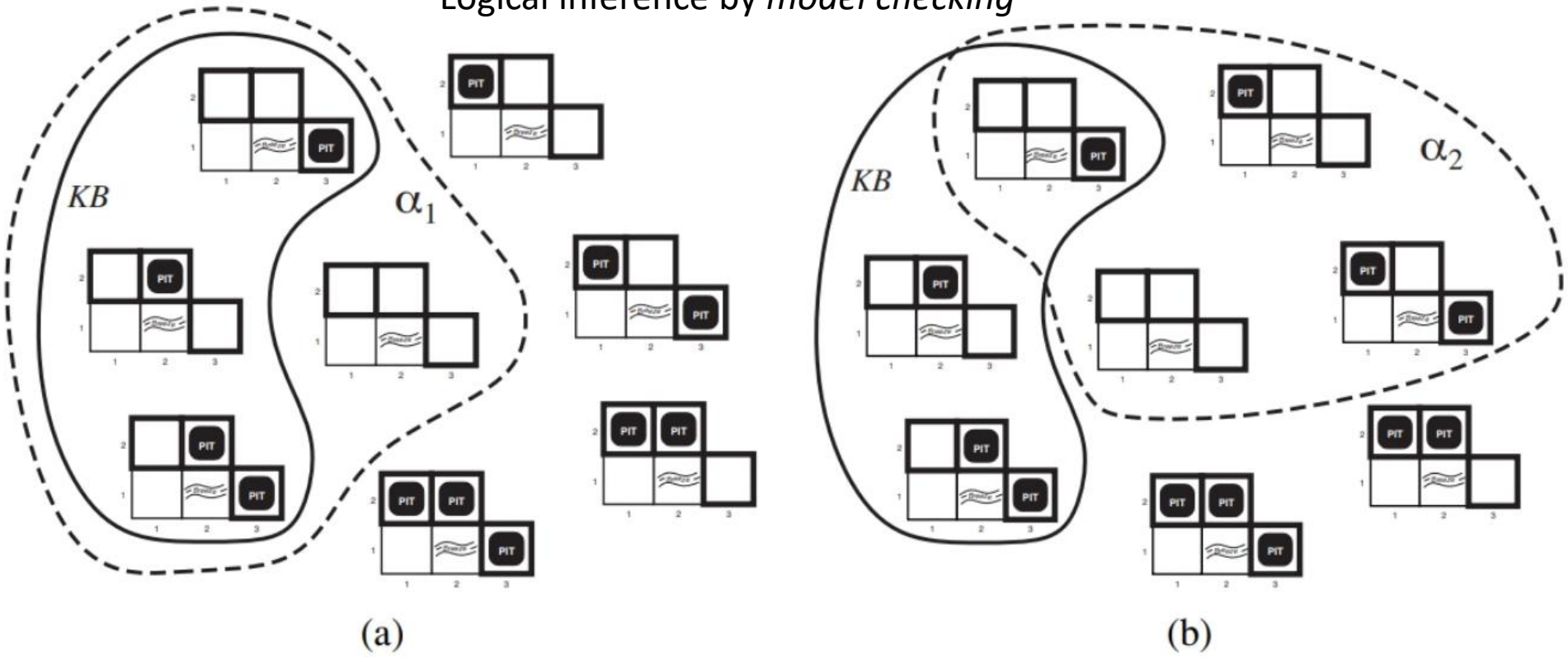
- $x = 0 \vDash xy = 0$



all possible pits in black squares, 2^3 possibilities…
KB shows what we know based on rules & percepts

RETURN TO WUMPUS WORLD

Logical inference by *model checking*

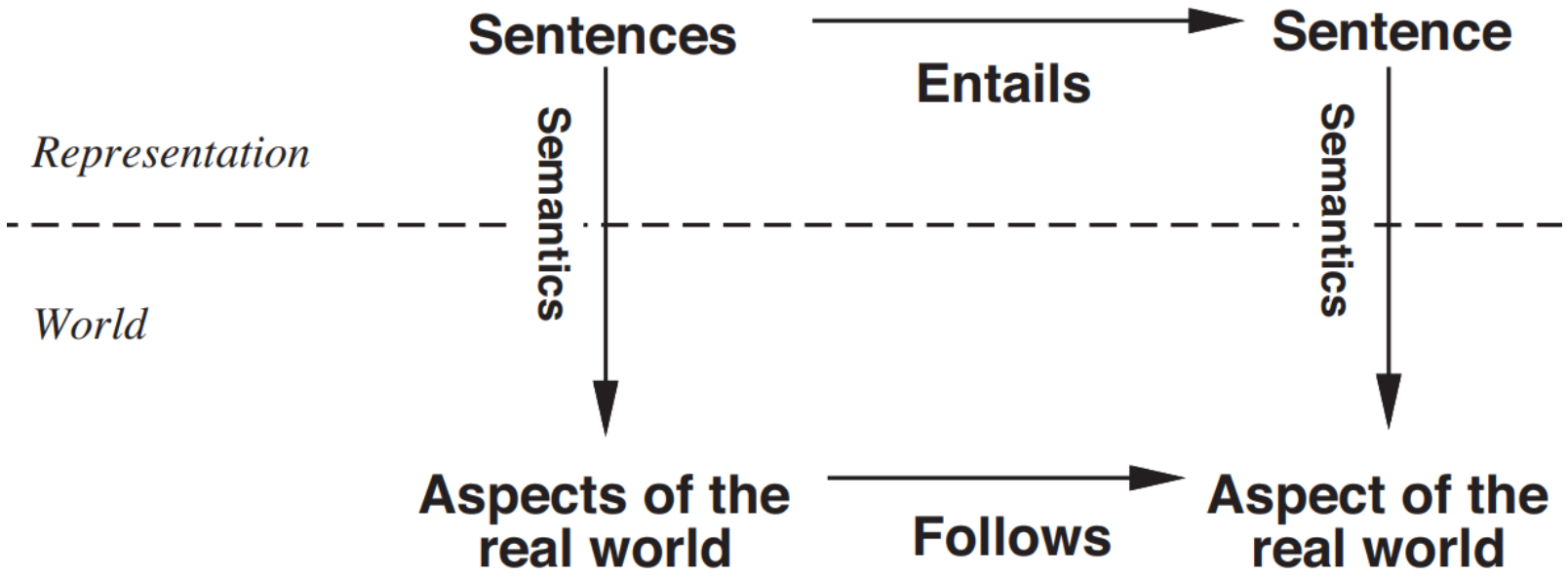


$$\alpha_1 = \text{No pit at } [1,2] \qquad \alpha_2 = \text{No pit at } [2,2]$$

Does KB |= α_1? KB |= α_2?

Inference algorithms

- Semantics define rules for determining truth of a sentence with respect to a particular model



Sentence construction: Propositional Logic

Propositional logic is for the most part the logic you learned to program with:

Sentence → AtomicSentence | ComplexSentence

AtomicSentence → true|false|Literal

ComplexSentence → (Sentence) | [Sentence]

| ¬ Sentence	*negation*	¬ (not)
| Sentence ^ Sentence	*conjunction*	^ (and)
| Sentence v Sentence	*disjunction*	v (or)
| Sentence ⇒ Sentence	*implication*	⇒ (implies)
| Sentence ⇔ Sentence	*biconditional*	⇔ (if and only if)

operator priority of logical connectives is shown by order,
e.g. (A v B ^ C) implies (A v (B ^ C)), literal is a propositional symbol e.g. A. same as ordinary arithmetic -2+4

Sentence semantics

- Sentences are reduced to true or false with respect to a specific model.
- In the Wumpus cave
  - We might denote the presence or absence of a pit by literals indexed by location: $P_{x,y}$
  - Example: $P_{1,2}$, $P_{2,2}$, and $P_{3,1}$ that have true/false values in any given model.
- Models must specify values for each proposition.
- To resolve a sentence: apply logical connectives to truth values (see Fig. 7.8 for truth tables)

Sentence semantics

- To resolve a sentence: apply logical connectives to truth values (see Fig. 7.8 for truth tables)

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

Figure 7.8 Truth tables for the five logical connectives. To use the table to compute, for example, the value of $P \vee Q$ when P is true and Q is false, first look on the left for the row where P is *true* and Q is *false* (the third row). Then look in that row under the $P \vee Q$ column to see the result: *true*.

Knowledge base rules

In the wumpus cave,

- denote pit & wumpus presence/absence by $P_{i,j}$ and $W_{i,j}$
- there is a breeze if and only if there is a neighboring pit:

$$B_{2,2} \Leftrightarrow P_{2,1} \vee P_{2,3} \vee P_{1,2} \vee P_{3,2}$$
$$B_{1,1} \Leftrightarrow P_{1,2} \vee P_{2,1}$$

- there is a stench iff the Wumpus is in the next cavern

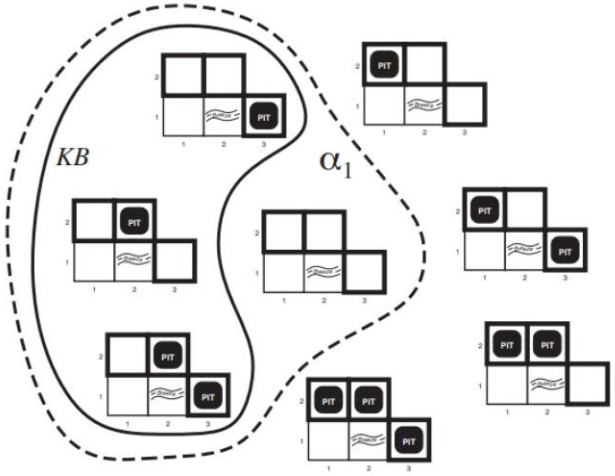
$$S_{2,2} \Leftrightarrow W_{2,1} \vee W_{2,3} \vee W_{1,2} \vee W_{3,2}$$
$$S_{1,1} \Leftrightarrow W_{1,2} \vee W_{2,1}$$

Stench and breeze sentences are true in *all* wumpus worlds.

Percepts

- There is no pit at 1,1
  $\neg P_{1,1}$
- There is no breeze at 1,1
  $\neg B_{1,1}$
- There is a breeze at 2,1
  $B_{2,1}$

Wumpus truth table



(a)

The sentences we write will suffice to derive $\neg P_{1,2}$ (there is no pit in [1,2]), as was done informally in Section 7.3. We label each sentence R_i so that we can refer to them:

- There is no pit in [1,1]:

  $R_1 : \quad \neg P_{1,1}$.

- A square is breezy if and only if there is a pit in a neighboring square. This has to be stated for each square; for now, we include just the relevant squares:

  $R_2 : \quad B_{1,1} \quad \Leftrightarrow \quad (P_{1,2} \vee P_{2,1})$.

  $R_3 : \quad B_{2,1} \quad \Leftrightarrow \quad (P_{1,1} \vee P_{2,2} \vee P_{3,1})$.

- The preceding sentences are true in all wumpus worlds. Now we include the breeze percepts for the first two squares visited in the specific world the agent is in, leading up to the situation in Figure 7.3(b).

  $R_4 : \quad \neg B_{1,1}$.

  $R_5 : \quad B_{2,1}$.

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	*false*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*false*	*false*
false	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*false*	*true*	*false*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	*true*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*false*	*true*	*true*	*false*
false	*true*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*false*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*true*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	*true*	*true*	*true*	*true*	*true*	*true*	*false*	*true*	*true*	*false*	*true*	*false*

Figure 7.9 A truth table constructed for the knowledge base given in the text. *KB* is true if R_1 through R_5 are true, which occurs in just 3 of the 128 rows (the ones underlined in the right-hand column). In all 3 rows, $P_{1,2}$ is false, so there is no pit in [1,2]. On the other hand, there might (or might not) be a pit in [2,2].

Simple entailment algorithm

-Symbol: basic atomic unit
(proposition, constant, variable, …)

```
def TruthTable-Entails(KB, α):
   symbols = proposition-symbols in KB and α
   return TT-Check-All(KB, α, symbols, {})

def TT-Check-All(KB, α, symbols, model):
   if empty(symbols):
      if pl-true(kb, model):  # Does the KB entail the model?
         return pl-true(α, model)  # Does α entail the model?
      else return True  # return true when KB does not hold
    else:
      # recursively enumerate the models
      (s, others) = (first(symbols), rest(symbols))
      return TT-Check-ALL(KB, α, others, model U {s=True}) and
             TT-Check-ALL(KB, α, others, model U {s=False})
```

inputs: KB, the knowledge base, a sentence in propositional logic
α, the query, a sentence in propositional logic

$symbols \leftarrow$ a list of the proposition symbols in KB and α
return TT-CHECK-ALL($KB, \alpha, symbols, \{\}$)

Figure 7.10 A truth-table enumeration algorithm for deciding propositional entailment. (TT stands for truth table.) PL-TRUE? returns *true* if a sentence holds within a model. The variable *model* represents a partial model—an assignment to some of the symbols. The keyword "**and**" is used here as a logical operation on its two arguments, returning *true* or *false*.

pl-true(a,b) is true if sentence a holds in model b

Concrete example:

- Knowledge base: $\neg P \vee Q, Q \Rightarrow M$
- Does $\neg P \vee Q, Q \Rightarrow M \vDash P \vee Q \vee M$?

P	Q	M	$\neg P \vee Q$	$Q \Rightarrow M$	$P \vee Q \vee M$
F	F	F	T	T	F
F	F	T	T	T	T
F	T	F	T	F	T
F	T	T	T	T	T
T	F	F	F	T	T
T	F	T	F	T	T
T	T	F	T	F	T
T	T	T	T	T	T

Concrete example:

- Knowledge base: $\neg P \vee Q$
- Does $\neg P \vee Q \vDash \neg(P \wedge \neg Q) \vee R$?

P	Q	R	$\neg P \vee Q$	$\neg(P \wedge \neg Q) \vee R$
F	F	F	T	T
F	F	T	T	T
F	T	F	T	T
F	T	T	T	T
T	F	F	F	F
T	F	T	F	T
T	T	F	T	T
T	T	T	T	T

```
def TruthTable-Entails(KB= ¬P ∨ Q, Q ⇒ M, α = P ∨ Q ∨ M):
   symbols = proposition-symbols in KB and α  # P, Q, M
   return TT-Check-All(KB, α, {P, Q, M}, {})

def TT-Check-All(KB, α, symbols, model):
   if empty(symbols):
      if pl-true(kb, model):  # Does model hold on left side of entailment?
         return pl-true(α, model)  # Is α entail the model?
      else return True  # entailment not affected when KB does not hold
    else:
      # recursively enumerate the models
      (s, others) = (first(symbols), rest(symbols))
      return TT-Check-ALL(KB, α, others, model U {s=True}) and
             TT-Check-ALL(KB, α, others, model U {s=False})
```

```
TT-Check-All(KB, α, {Q, M}, {P=T})
TT-Check-All(KB, α, {Q, M}, {P=F})
```

eventually everything is assigned to a model, e.g. {P=T,Q=T,M=T}

if pl-true($\{\neg P \vee Q, Q \Rightarrow M\}$, $\{P = T, Q = T, M = T\}$)
   return pl-true($\{P \vee Q \vee M\}$,$\{P = T, Q = T, M = t\}$)
   else return True

Simple entailment algorithm

Summary of model checking

- Recursively generates all possible combinations of truth values for every symbol.
- Checks if the knowledge base holds true for a world model (symbol value assignment)
  - If so, returns true/false showing whether sentence α also holds on model.
  - Otherwise returns true as we don't care about whether α holds outside the KB (implies).

Can we prove things without enumerating everything? - Using inference

- Typically more efficient as we do not have to enumerate every possible value.

- Can be formulated as a search:
  - Initial State – Initial knowledge base
  - Actions – Inference rules applied to sentences
  - Results – Action result is the sentence rewritten by the inference rule
  - Goal – State containing the sentence we are trying to prove

Inference rules – Chain of conclusions leading to desired goals

- Notation for rewriting logic

$$\frac{Sentence_1, Sentence_2}{Sentence_3}$$ means sentences 1 and 2 imply 3

- Rules
  - *Modus ponens*

    (latin for model that affirms)

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta} \qquad \frac{\text{breeze} \Rightarrow \text{pit in neighboring cavern}, \text{breeze}}{\text{pit in neighboring cavern}}$$

  - And elimination

$$\frac{\alpha \wedge \beta}{\alpha} \qquad \frac{\text{glitter} \wedge \text{breeze} \wedge \text{stench}}{\text{glitter}}$$

Notation and inference rules

- Rules
  - Biconditional elimination

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$$

$$\frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

Note that we can also write these as equivalences

$$\text{e.g. } \alpha \Leftrightarrow \beta \equiv (\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$$

Inference rules

- Commutativity, associativity, and distributivity for $\wedge$ and $\vee$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$$

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$$

Inference rules

- Double-negation elimination

$$\neg(\neg\alpha) \equiv \alpha$$

- Contraposition

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$$

- Implication elimination

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$$

- DeMorgan's rule

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$$

Proof by hand

- Knowledge base

$$\neg P_{1,1},\ \ B_{1,1} \Leftrightarrow P_{1,2} \vee P_{2,1},\ \ B_{2,1} \Leftrightarrow P_{1,1} \vee P_{2,2} \vee P_{3,1}$$

- Percepts (added to the knowledge base)

$$\neg B_{1,1}, B_{2,1}$$

- Does the KB entail the lack of a pit at 1,2?

$B_{1,1} \Leftrightarrow P_{1,2} \vee P_{2,1}$ from KB

$(B_{1,1} \Rightarrow P_{1,2} \vee P_{2,1}) \wedge (P_{1,2} \vee P_{2,1} \Rightarrow B_{1,1})$ biconditional elim.

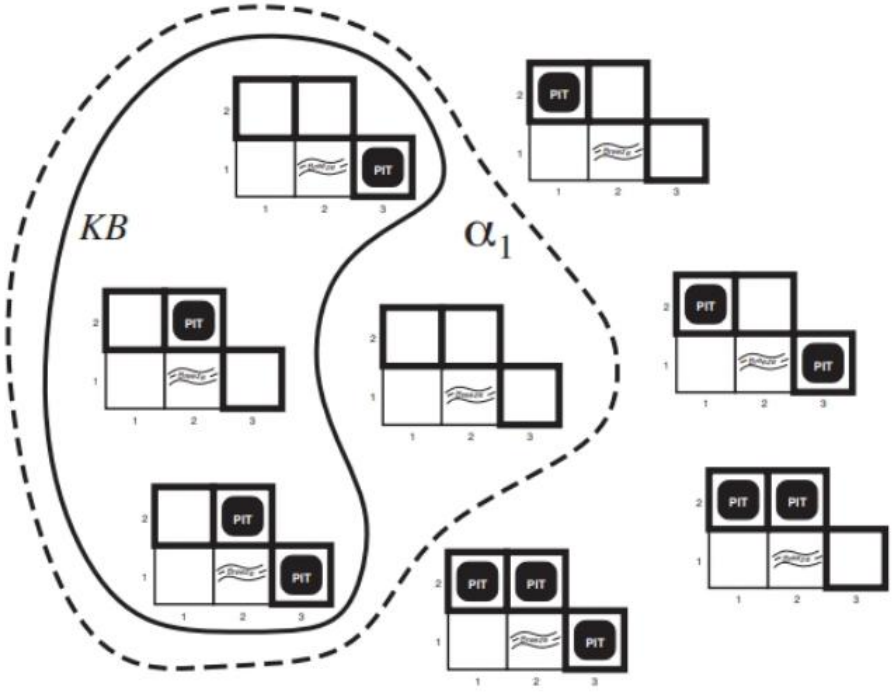
$P_{1,2} \vee P_{2,1} \Rightarrow B_{1,1}$ and elimination

$\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})$ contrapositive

$\neg(P_{1,2} \vee P_{2,1})$ modus ponens from percept $\neg B_{1,1}$

$\neg P_{1,2} \wedge \neg P_{2,1}$ DeMorgan's Rule

$\neg P_{1,2}$ and elimination ■ $KB \vDash (\alpha = \neg P_{1,2})$



(a)

Can we prove things without enumerating everything? - Proof by contradiction

Calvin & Hobbes ©Bill Watterson used under educational fair use

- Deduction theorem

For arbitrary sentences α and β,
$\alpha \vDash \beta$ iff $(\alpha \Rightarrow \beta)$ is valid.

- So we try to show that $\neg(\alpha \Rightarrow \beta)$ is not satisfiable:

$$\neg(\alpha \Rightarrow \beta)$$

$\neg(\neg\alpha \vee \beta)$ by implication elimination $\qquad (\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$

$\alpha \wedge \neg\beta$ by deMorgan's rule $\qquad (\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$

- If we can show that $\alpha \wedge \neg\beta$ leads to a contradiction and hence not satisfiable, then $\alpha \vDash \beta$.

Conjunctive normal form

All propositional logic sentences can be transformed to conjunctions of disjunctive clauses

- Formula (… OR … OR …) AND (… OR …) AND …

- Clause: A disjunction of one or more literals
- Literal: A symbol or its negation
- Symbol: basic atomic unit (proposition, constant, variable, …)

$$
\begin{aligned}
CNFSentence &\rightarrow Clause_1 \wedge \cdots \wedge Clause_n \\
Clause &\rightarrow Literal_1 \vee \cdots \vee Literal_m \\
Literal &\rightarrow Symbol \mid \neg Symbol \\
Symbol &\rightarrow P \mid Q \mid R \mid \ldots
\end{aligned}
$$

Breeze/Pit Example

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$
$$\left(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})\right) \wedge \left((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}\right)$$
$$\left(\neg B_{1,1} \vee (P_{1,2} \vee P_{2,1})\right) \wedge \left(\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1}\right)$$
$$\left(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}\right) \wedge \left((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1}\right)$$
$$\left(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}\right) \wedge \left(\neg P_{1,2} \vee B_{1,1}\right) \wedge \left(\neg P_{2,1} \vee B_{1,1}\right)$$

$$\frac{(P_{1,1} \vee P_{3,1}) \wedge (\neg P_{1,1} \vee \neg P_{2,2})}{P_{3,1} \vee \neg P_{2,2}}$$

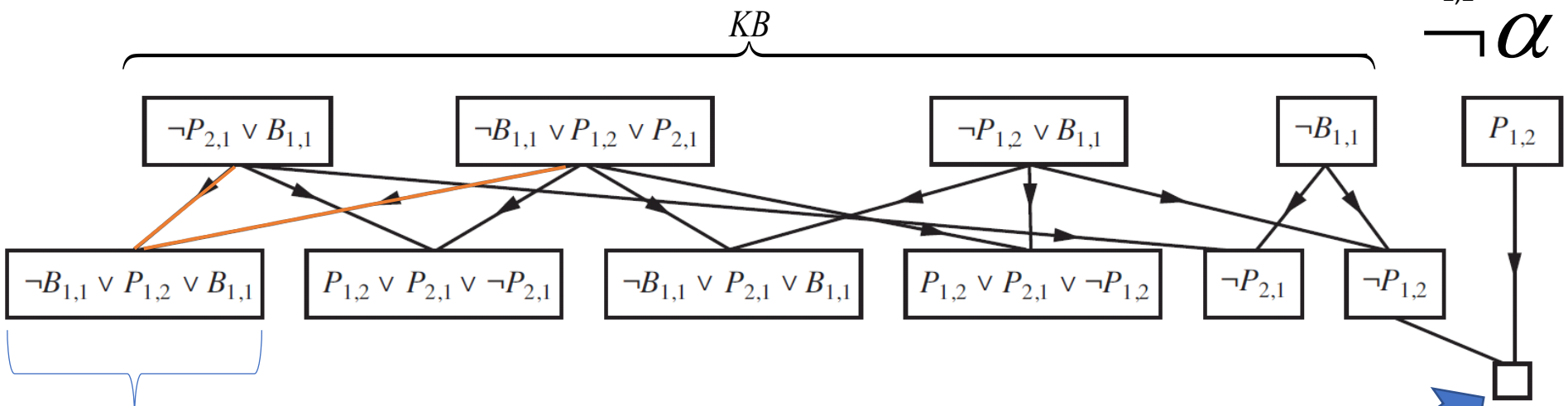
Resolution algorithm

- Permits us to show KB ⊨ α using only the resolution rule.
- All sentences must be in conjunctive normal form
- Resolution algorithm
  - Lets us show that something is satisfiable, that a model exists where a proposition is true
  - We want to show KB |= α, but the resolution algorithm only shows us satisfiability.
  - Fortunately, $KB \vDash \alpha \Leftrightarrow sentence\ KB \wedge \neg\alpha\ is\ unsatisfiable$
    so if we assume α is false and show this leads to a contradiction with know KB
    we have a proof by contradiction.

There is no pit $\alpha = \neg P_{1,2}$

Try to prove that there is a pit $P_{1,2}$

$\neg\alpha$

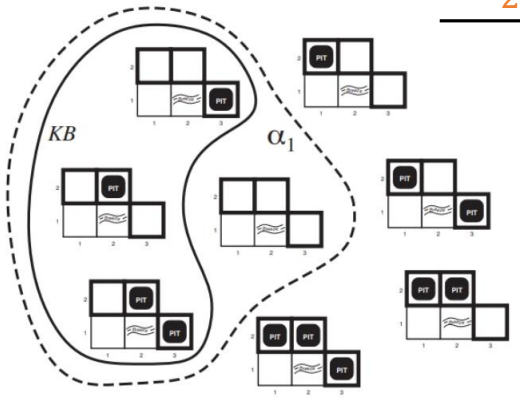


$$\frac{\neg P_{2,1} \vee B_{1,1},\ \neg B_{1,1} \vee P_{1,2} \vee P_{2,1}}{\neg B_{1,1} \vee P_{1,2} \vee B_{1,1}}$$

$$\frac{\neg P_{1,2},\ P_{1,2}}{\emptyset}$$

Contradiction!

We sometimes denote this ϵ



(a)

Applications of the resolution rule, Fig. 7.13

Resolution algorithm

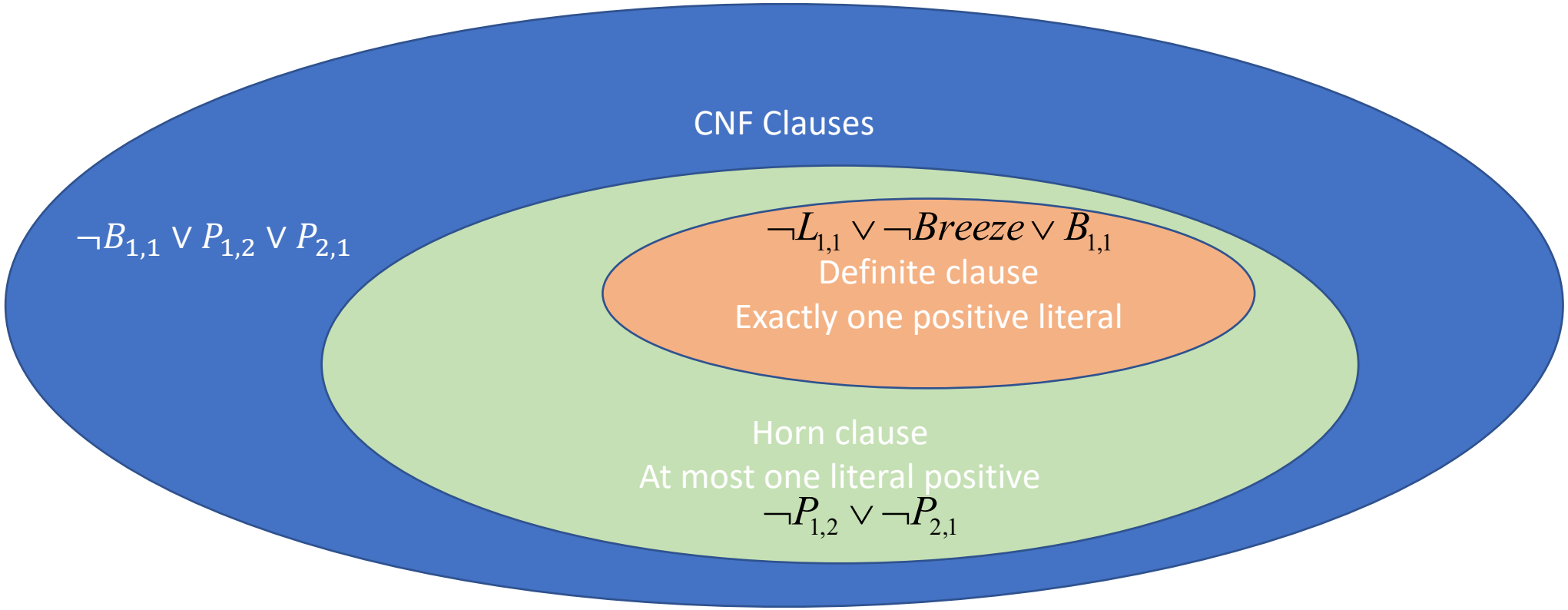
A simple resolution algorithm for propositional logic. The function PL-Resolve returns the set of all possible clauses obtained by resolving its two inputs

```
def PL-Resolution(KB, α)
    clauses = set of conjunctive normal form clauses of KB ∧ ¬α

    new = {}
    while True:
        for each distinct pair with complementary literals Ci, Cj ∈ clauses:
            resolvents = PL-Resolve(Ci, Cj)
            if ∅∈resolvents, return True  #could not satisfy, contradiction
            new = new U resolvents
        if new ⊆ clauses, return False   # everything satisfied
        clauses = clauses U new
```

Restricted conjunctive normal form (CNF) clauses

Full power of resolution is not always needed



DeMorgan's rule $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$

Implication elimination $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$

Restricted CNF clauses

- Definite and Horn clauses are interesting as there are more efficient resolution techniques.
- Many problems can be posed as Horn or definite clause problems:

- Definite Horn Clause: $P \vee \neg Q_1 \vee \neg Q_2 \vee \neg Q_3 \vee \ldots \vee \neg Q_n$

- Logically equivalent: implication $(Q_1 \wedge Q_2 \wedge Q_3 \wedge \ldots \wedge Q_n) \rightarrow P$
- Meaning "if all of Q1, Q2, …, Qn are true, P must be true"

DeMorgan's rule $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$

Implication elimination $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$

Restricted CNF clauses

- Many problems can be posed as Horn or definite clause problems:

$$\neg L_{1,1} \vee \neg Breeze \vee B_{1,1} \equiv (L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$$

- Horn clauses are usually written as implications and have names for clause parts.
  - premise: body
  - head: conclusion
  - fact: sentences

  consisting of single positive literal,

$$\underbrace{Breeze}_{fact} \qquad \underbrace{(L_{1,1} \wedge Breeze)}_{body} \Rightarrow \underset{head}{B_{1,1}}$$

$$\equiv True \Rightarrow Breeze$$

meaning if all facts are true, then conclusion must be true

Why care about Horn clauses?

- Set of possible clauses in resolution closure of KB and α may be large
- We can prove Horn clause propositions in time that is linear with respect to the size of KB

Road Runner fan art dohtem.com

Restricted CNF Clauses

- A Horn clause with no positive literal is called a *goal* clause.
- Efficient inference with Horn clauses is done through chaining.
  - forward-chaining – Data-driven, use known precepts to drive resolution of graph constructed from Horn clauses.
  - backward-chaining – Query driven, search and-or graph for goal.

- The language *Prolog* (Programming in Logic) is a Horn clause processing language.

$$\neg L_{1,1} \vee \neg Breeze \vee B_{1,1} \equiv (L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$$

Horn clauses as graphs

Example of a simple knowledge base of Horn clauses

- with A and B are known facts
- Begin with A and B, if all premise of implication are known,
  - conclusion added to the knowledge base

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

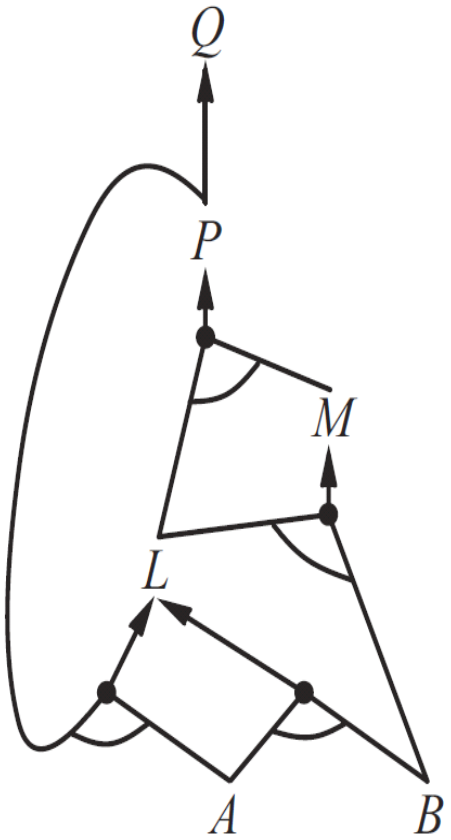
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

$$A$$

$$B$$



Forward chaining

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

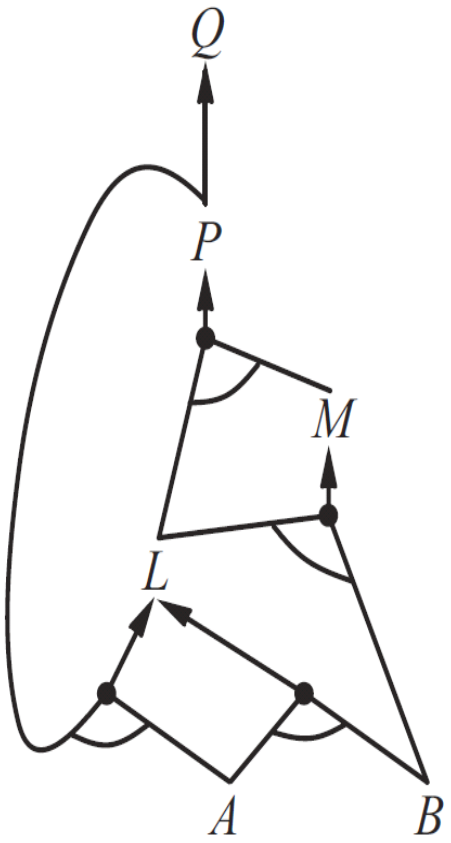
$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B




```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional definite clauses
          q, the query, a proposition symbol
  count ← a table, where count[c] is the number of symbols in c's premise
  inferred ← a table, where inferred[s] is initially false for all symbols
  agenda ← a queue of symbols, initially symbols known to be true in KB

  while agenda is not empty do
     p ← POP(agenda)
     if p = q then return true
     if inferred[p] = false then
        inferred[p] ← true
        for each clause c in KB where p is in c.PREMISE do
           decrement count[c]
           if count[c] = 0 then add c.CONCLUSION to agenda
  return false
```

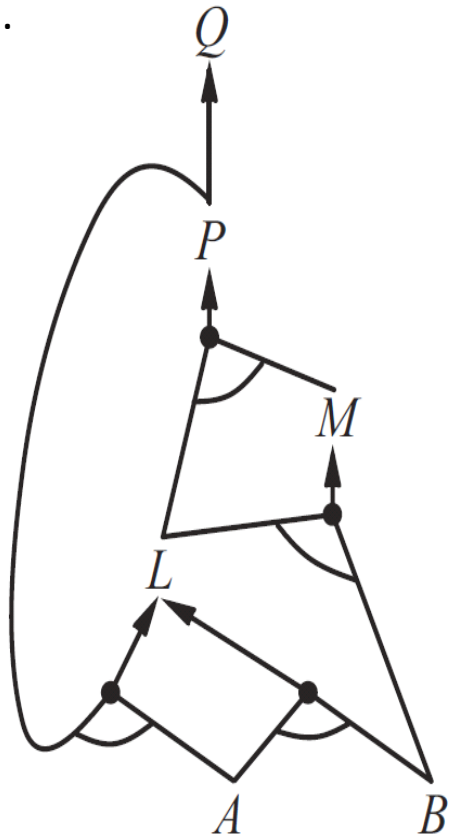
Prove L from A & B, continue propagating

Algorithm on p. 258, but you need only understand this intuitively.

Backward chaining

Goal directed reasoning – start with goal
Usually faster…

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$



Start with query, e.g. Q and perform AND-OR search

- AND node: all child nodes explored simultatnously
- OR node: one of the child nodes must be selected

Backward chaining: Prolog

code

```
% Facts
eats(john, apples).
eats(mary, bananas).
likes(mary, music).

% Rule
likes(john, X) :- eats(X, bananas).
```

query

```
?- likes(john, mary).
```

Backward Chaining Process:

- **Goal**: query `likes(john, mary).`
- **Rule Match**: finds the rule `likes(john, X) :- eats(X, bananas)`. To satisfy `likes(john, mary)`, Prolog tries to unify `mary` with `X` in the rule's head, which means `X` becomes `mary`.
- **Subgoal**: Now, Prolog needs to prove the rule's body with `X` instantiated to `mary`, which becomes the new subgoal: `eats(mary, bananas).`
- **Facto Match**: Prolog searches for `eats(mary, bananas)` and finds it as a fact in the knowledge base.
- Since the subgoal `eats(mary, bananas)` is proven, the original goal `likes(john, mary)` is also proven

Putting it all together
Logical Agents

- Begin with knowledge base.
  For the Wumpus world
  - big long list of pits, breezes, and stenches (oh my):

$$B_{1,1} \Leftrightarrow P_{1,2} \vee P_{2,1}$$
$$S_{1,1} \Leftrightarrow W_{1,2} \vee W_{2,1}$$
$$B_{2,2} \Leftrightarrow P_{2,1} \vee P_{2,3} \vee P_{1,2} \vee P_{3,2}$$
$$S_{2,2} \Leftrightarrow W_{2,1} \vee W_{2,3} \vee W_{1,2} \vee W_{3,2}$$
$$\ldots$$

Tedious…. first order logic provides
a better way to do this Chapters 8 & 9

  - what we know about starting square and that we have an arrow

$$\neg W_{1,1} \wedge \neg P_{1,1} \wedge HaveArrow$$

Logical Agents

- Wumpus world continued
  - what we know about the wumpus:
    - At least one $W_{1,1} \vee W_{1,2} \vee W_{1,3} \vee W_{1,4} \vee W_{2,1} \vee \ldots \vee W_{4,4}$
    - No more than one – for every pair of locations at least one of them must not have a wumpus

$$\neg W_{1,1} \vee \neg W_{1,2}$$
$$\neg W_{1,1} \vee \neg W_{1,3}$$
$$\neg W_{1,1} \vee \neg W_{1,4}$$
$$\neg W_{1,1} \vee \neg W_{2,1}$$
$$\neg W_{1,1} \vee \neg W_{2,2}$$
$$\ldots$$
$$\neg W_{4,4} \vee \neg W_{4,3}$$

Percepts

- How do we incorporate percepts?
  - We might observe *Stench* at time 6, but not at time 7.
  - Remember, KB cannot change its mind! PL (propositional logic) is *monotonic.*
  - We can modify MAKE-PERCEPT-SENTENCE to create distinct literals with time names: $Stench^6$ $\neg Stench^7$

Percepts

- We now have two types of variables:
  - Timestamped variables called *fluents*
    - latin fluens, flowing, to refer an aspect of the world that changes
  - and *atemporal* variables that do not vary with time.

Should HaveArrow be a fluent or atemporal variable?

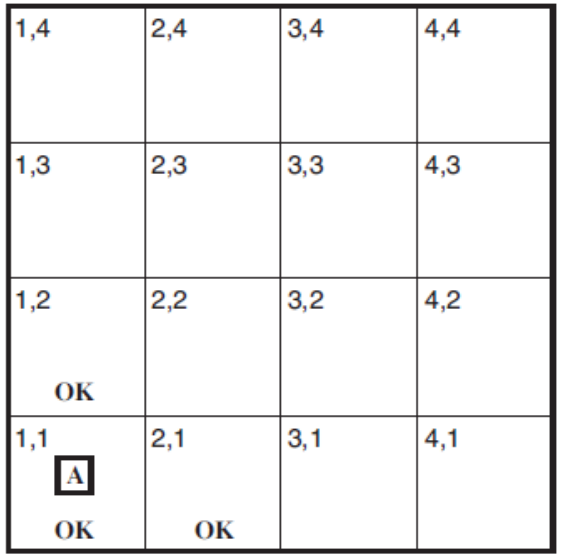
How about $P_{3,1}$

Actions and transition models

- Suppose agent initially at 1,1 ($L^0_{1,1}$) facing east. An *effect axiom* (general knowledge about how the world works) could be added to model moving forward*:*

$$L^0_{1,1} \wedge FacingEast^0 \wedge Forward^0 \Rightarrow (L^1_{2,1} \wedge \neg L^1_{1,1})$$

- Other effect axioms could be added for:
  - grab, shoot, climb, turnleft, turnright.



Successor-state axiom example

- Hunter is at $L_{1,1}$ at time t+1:

$$L_{1,1}^{t+1} \Leftrightarrow (L_{1,1}^{t} \wedge (\neg Forward^{t} \vee Bump^{t+1})) \\ \vee (L_{1,2}^{t} \wedge (South^{t} \wedge Forward^{t})) \\ \vee (L_{2,1}^{t} \wedge (West^{t} \wedge Forward^{t})$$

- Hunter has arrow at time t+1:

$$HaveArrow^{t+1} \Leftrightarrow (HaveArrow^{t} \wedge \neg Shoot)$$

Logical Agents

CARD GAME
ROYAL FLUSH 1895

- Pure logic agents simply try to prove a goal
  - Wumpus example sentence contains clauses for:
    - $Init^0$ – assertions about initial state
    - $Transition^1$, … $Transition^t$, successor-state axioms for all possible actions times 1,2,…,t.
    - Goal: $HaveGold^t$ ^ $ClimbedOut^t$
  - If a model satisfies the clause, we can look at the action variables at each time and we have our plan.
  - Otherwise not possible in t steps.

Caveat: Not for partially observable environments, solver will simply set variables of hidden information to satisfy the problem

Hybrid Wumpus agent (partial)

Hybrid agents - Use combination of logic, search and rule-based actions.

```
hybrid-wumpus-agent(perceptlist, t) {
   Tell(KB, Make-Percept-Sentence(perceptlist, t))
   if t > 0:
      Tell(KB, successor state axioms for time t)
   safe = {[x,y] : Ask(KB, Ok^t_xy) = True}   # list of known safe caves

   # check goals by priority
   if Ask(KB, Glitter^t) = True:
      plan = [Grab] + PlanRoute(current, {[1,1]}, safe) + [Climb]
   else:
      unvisited = {[x,y] : Ask(KB, L^t'_x,y) = false ∀t'≤t }   # places we haven't been
      plan = PlanRoute(current, unvisited∩safe, safe)
   if not plan and Ask(KB,HaveArrow^t) == True:
      # no glitter or way to a safe square
      possiblewumpus = {[x,y] : Ask(KB, ¬W_x,y)==false}
      plan = plan-shot(current, possiblewumpus, safe)
```

Hybrid Wumpus agent (partial)

```
    if not plan:
        # Couldn't move anywhere or shoot, take a risk
        mightbesafe = {[x,y] : Ask(KB, ¬OK^t_{x,y}) == False}
        plan = PlanRoute(current, unvisited ∩ mightbesafe, safe)
    if not plan:
        # Can't get anywhere safely.  Call it a day
        plan = PlanRoute(current, {[1,1]}, safe) + [Climb]

    action = pop(plan)
    Tell(KB, Make-Action-Sentence(action, t))
    t = t + 1
    return action

PlanRoute(current, goals, allowed)
    # Plan a path from current to any goal through the set
    # of allowed nodes
    problem = RouteProblem(current, goals, allowed)
    return A* graph search(problem)
```